Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 6

Disciplina: Tehnici de programare

Tema: Forensics și steganografie

A efectuat : st. gr.SI – 264: Railean Adrian

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

1. Sarcina lucrării

Scopul acestui laborator este însușirea și practicarea conceptelor fundamentale de analiză digitală criminalistică (forensics), criptanaliză și steganografie pe imagini în limbajul Python. Lucrarea are ca obiective:

1. Recunoașterea și decodarea automată a formatelor multiple de codificare (Base64, Hex, URL).

2. Decodarea succesivă a mesajelor ascunse sub mai multe straturi de codificare.

3. Realizarea unui atac prin forță brută pentru decriptarea unui mesaj protejat prin cifrul simetric XOR pe 1 octet.

4. Spargerea hash-urilor criptografice (MD5, SHA-256) prin atac de dicționar.

5. Automatizarea funcționalităților printr-o interfață CLI (argparse), salvare JSON și validarea prin teste unitare (pytest).

6. Identificarea tipului real al fișierelor prin inspectarea semnăturilor binare (Magic Bytes).

7. Extragerea textului imprimabil din fișiere binare și analizarea metadatelor EXIF/GPS pentru geolocalizare.

8. Decuparea fișierelor ascunse în structura imaginilor prin tehnica de File Carving.

9. Ascunderea și recuperarea unui mesaj secret într-o imagine utilizând tehnica steganografică LSB.

2. Executarea lucrării

2.1 Decodarea succesivă a mesajului cu straturi multiple

În fișierul decode.py s-au implementat funcțiile pentru identificarea formatelor de codificare și decodarea succesivă a straturilor din probe/mesaj.txt până la obținerea textului în clar.

```
===== Exercitiul 4 =====
Pas 1: strat base64 -> 53595759
Pas 2: strat hex -> SYWY
Mesaj final: SYWY
```

Figura 2.1 – Decodarea succesivă a mesajului cu straturi multiple (probe/mesaj.txt)

2.2 Criptanaliză XOR prin forță brută pe 1 octet

S-a realizat atacul prin forță brută pe 256 de chei posibile asupra fișierului probe/xor.bin, identificând cheia corectă pe baza lizibilității caracterelor și recuperând mesajul secret.

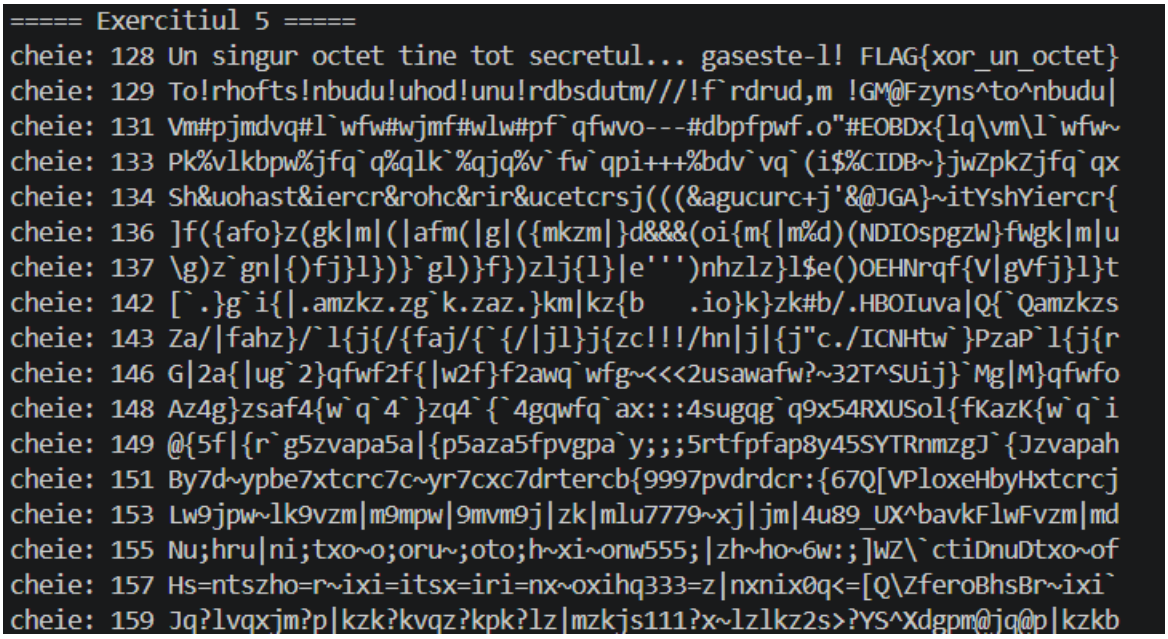


Figura 2.2 – Rularea atacului prin forță brută și identificarea cheii XOR 128

2.3 Spargerea hash-urilor criptografice prin dicționar

S-a implementat funcția de spargere a hash-urilor pe baza listei de cuvinte probe/wordlist.txt, fiind identificată parola în clar asociată amprentei MD5 din probe/hashuri.txt.

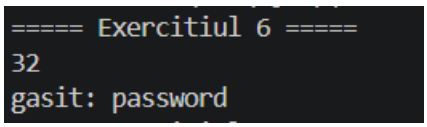


Figura 2.3 – Spargerea hash-ului MD5 prin atac de dicționar

2.4 Validarea automată a decodorului prin teste unitare (pytest)

A fost rulată suita de teste automate din teste/test_decode.py, verificând corectitudinea funcțiilor de detecție, decodare multi-strat, atac XOR și verificare hash.

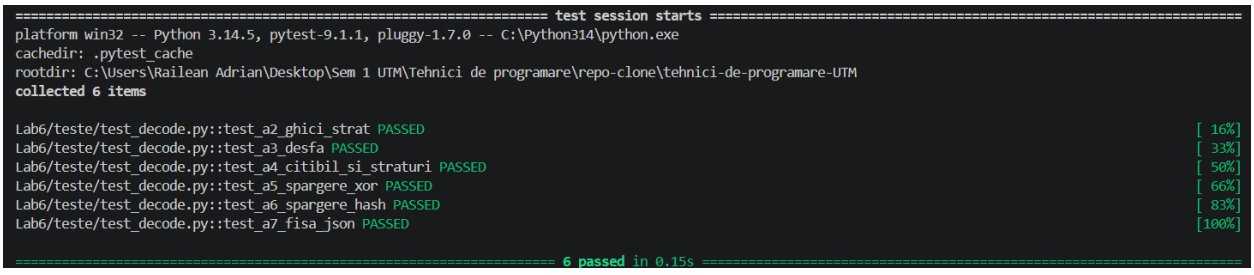


Figura 2.4 – Rularea suitei de teste unitare (pytest teste/test_decode.py)

2.5 Identificarea formatului real prin Magic Bytes

În fișierul forensics.py s-au citit primii octeți din antetul fișierelor din folderul probe/ pentru a valida formatele reale JPEG și PNG, indiferent de extensie.

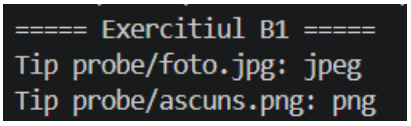


Figura 2.5 – Determinarea tipului real al fișierelor prin Magic Bytes

2.6 Extragerea secvențelor de text din fișiere binare (strings)

S-a realizat extragerea secvențelor de caractere imprimabile din fișierul probe/ascuns.png, fiind identificată structura internă a imaginii și prezența unei arhive comprimate.

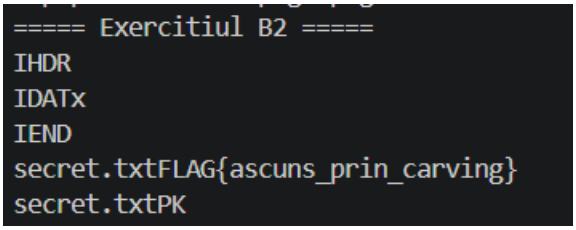


Figura 2.6 – Extragerea secvențelor de text imprimabil din fluxul binar

2.7 Extragerea metadatelor EXIF și conversia coordonatelor GPS

Cu ajutorul bibliotecii Pillow s-au extras metadatele EXIF din probe/foto.jpg, iar coordonatele GPS au fost convertite în grade zecimale pentru localizarea exactă a campusului UTM.

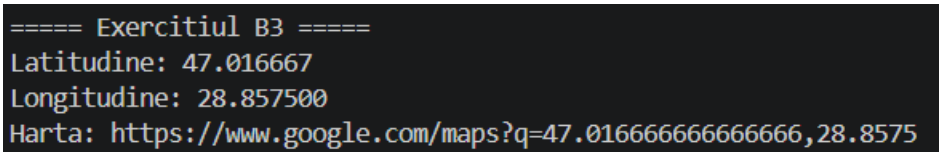


Figura 2.7 – Extragerea metadatelor EXIF și geolocalizarea pe hartă

2.8 Recuperarea arhivei ascunse prin File Carving

Prin căutarea semnăturii ZIP după marcatorul terminal al imaginii PNG, s-a decupat și extras arhiva gasit.zip, recuperând conținutul fișierului secret.txt.

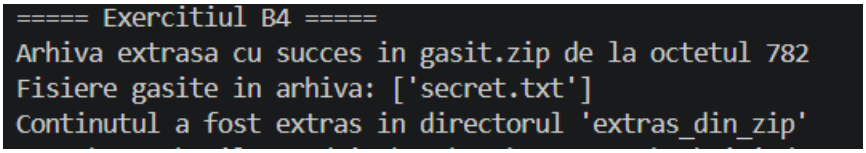


Figura 2.8 – Extragerea arhivei ZIP prin File Carving din imaginea PNG

2.9 Steganografie LSB: Ascunderea și recuperarea mesajului

În modulul stego.py s-au definit funcțiile pentru inserarea mesajului secret în biții cel mai puțin semnificativi ai imaginii probe/foto.jpg și recuperarea acestuia din fișierul rezultat stego.png.

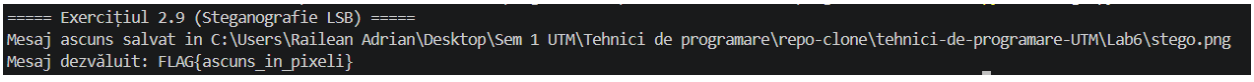


Figura 2.9 – Inserarea și recuperarea mesajului steganografic LSB

2.10 Generarea fișierului de sinteză fisa.json

Rularea decodorului cu parametrul de intrare a decodificat șirul Base64 și a salvat datele extrase în formatul structurat fisa.json.

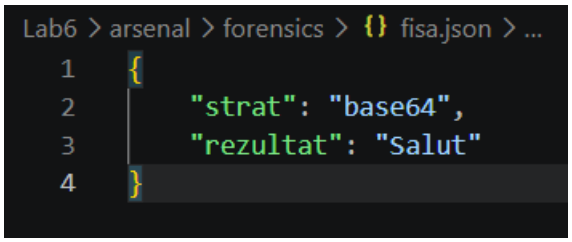


Figura 2.10 – Conținutul fișierului structurat arsenal/forensics/fisa.json

3. Concluzii

În cadrul acestui laborator am aprofundat conceptele fundamentale de analiză digitală criminalistică (forensics), criptanaliză și steganografie în limbajul Python. Am învățat să inspectez fișierele la nivel de octeți bruti, să determin formatul autentic pe baza semnăturilor binare (Magic Bytes) și să aplic metode clasice de recuperare a datelor ascunse sau codificate.

De asemenea, am exersat scrierea de scripturi modulare și reutilizabile pentru atacuri de dicționar, atacuri prin forță brută pe cifrul XOR, decupare de date (File Carving) și manipulare directă a pixelilor prin steganografie LSB. Validarea funcționalităților prin suita de teste automate (pytest) și exportul centralizat în format JSON confirmă buna funcționare și robustețea instrumentelor implementate.

4. Bibliografie

1. Documentation Python 3 - Standard Library (hashlib, base64, urllib, zipfile, argparse): https://docs.python.org/3/library/

2. Pillow (PIL Fork) Documentation - Image Processing and EXIF Metadata: https://pillow.readthedocs.io/

3. Universitatea Tehnică a Moldovei, „Ghid de laborator: Tehnici de Programare — Lucrarea 6”, Chișinău, 2026.

5. Anexe

Anexa A. Codul fișierului arsenal/forensics/decode.py

```
import base64
from urllib.parse import unquote
import re
import os
import hashlib
import argparse
import json
from pathlib import Path

# Calea catre directorul Lab6 (doua nivele deasupra lui decode.py)
BASE_LAB = Path(__file__).resolve().parent.parent.parent

def ghici_strat(s):
    s_curat = s.strip()
    if "%" in s_curat:
        return "url"
    if len(s_curat) % 2 == 0 and bool(re.fullmatch(r"[0-9a-fA-F]+", s_curat)):
        return "hex"
    if len(s_curat) % 4 == 0 and bool(re.fullmatch(r"[A-Za-z0-9+/=]+", s_curat)):
        return "base64"
    return "necunoscut"

def desfa(s):
    strat = ghici_strat(s)
    s_curat = s.strip()

    if strat == "base64":
        rezultat = base64.b64decode(s_curat)
    elif strat == "hex":
        rezultat = bytes.fromhex(s_curat)
    elif strat == "url":
        rezultat = unquote(s_curat).encode()
    else:
        rezultat = s_curat.encode("utf-8")

    return strat, rezultat

def citibil(b):
    if not b:
        return False
    return all(32 <= c < 127 for c in b)

if __name__ == "__main__":
    os.chdir(BASE_LAB)

    print("===== Exercitiul 1 =====")
    s = "Salut"
    b = s.encode()
    print("Bytes:", b)
    print("Primul octet (b[0]):", b[0])
    h = b.hex()
    print("Hex:", h)
    b_refacut = bytes.fromhex(h)
    print("Bytes refăcut din hex:", b_refacut)
```

```
    print("===== Exercitiul 2 =====")
    print(ghici_strat("%2Fetc"))
    print(ghici_strat("48656c6c6f"))

    print("===== Exercitiul 3 =====")
    strat, rez = desfa("U2FsdXQ=")
    print(strat, rez)
    strat, rez = desfa("48656c6c6f")
    print(strat, rez)
    strat, rez = desfa("%2Fetc")
    print(strat, rez)

    print("===== Exercitiul 4 =====")
    val = "NTM1OTU3NTk="
    mesaj_final = val
    for pas in range(8):
        strat = ghici_strat(val)
        if strat == "necunoscut":
            mesaj_final = val
            break
        try:
            _, b = desfa(val)
            if not citibil(b):
                mesaj_final = val
                break
            text = b.decode("utf-8")
            print(f"Pas {pas + 1}: strat {strat} -> {text}")
            mesaj_final = text
            val = text
        except Exception:
            mesaj_final = val
            break
    print("Mesaj final:", mesaj_final)

    print("===== Exercitiul 5 =====")
    date = open(BASE_LAB / "probe" / "xor.bin", "rb").read()
    for k in range(256):
        clar = bytes(b ^ k for b in date)
        if citibil(clar):
            print("cheie:", k, clar.decode("utf-8"))

    print("===== Exercitiul 6 =====")
    tinta = open(BASE_LAB / "probe" / "hashuri.txt", encoding="utf-
8").readline().strip()
    print(len(tinta))
    for cuv in open(BASE_LAB / "probe" / "wordlist.txt", encoding="latin1"):
        cuv = cuv.strip()
        if hashlib.md5(cuv.encode()).hexdigest() == tinta:
            print("gasit:", cuv)
            break

    print("===== Exercitiul 7 =====")
    p = argparse.ArgumentParser()
    p.add_argument("intrare", nargs="?", default="U2FsdXQ=")
    p.add_argument("--fisier", action="store_true")
    a = p.parse_args()

    if a.fisier:
        continut = open(a.intrare, "r", encoding="utf-8").read().strip()
    else:
```

```
        continut = a.intrare

    strat, rez_bytes = desfa(continut)
    try:
        rez_text = rez_bytes.decode("utf-8")
    except UnicodeDecodeError:
        rez_text = rez_bytes.hex()

    date_salvare = {
        "strat": strat,
        "rezultat": rez_text
    }

    cale_fisa = Path(__file__).resolve().parent / "fisa.json"
    with open(cale_fisa, "w", encoding="utf-8") as f:
        json.dump(date_salvare, f, indent=4)

    print("Strat decodat:", strat)
    print(f"Salvat in {cale_fisa.name}:", rez_text)
```

Anexa B. Codul fișierului teste/test_decode.py

```
import sys
import os
import hashlib
import json
from pathlib import Path

# Adaugă automat directorul Lab6 în calea Python (sys.path)
BASE_DIR = Path(__file__).resolve().parent
while BASE_DIR.name != "Lab6" and BASE_DIR != BASE_DIR.parent:
    BASE_DIR = BASE_DIR.parent

sys.path.insert(0, str(BASE_DIR))

from arsenal.forensics.decode import ghici_strat, desfa, citibil

def test_a2_ghici_strat():
    assert ghici_strat("%2Fetc") == "url"
    assert ghici_strat("48656c6c6f") == "hex"
    assert ghici_strat("U2FsdXQ=") == "base64"
    assert ghici_strat("salut!") == "necunoscut"

def test_a3_desfa():
    strat_b64, rez_b64 = desfa("U2FsdXQ=")
    assert strat_b64 == "base64"
    assert rez_b64 == b"Salut"

    strat_hex, rez_hex = desfa("48656c6c6f")
    assert strat_hex == "hex"
    assert rez_hex == b"Hello"

    strat_url, rez_url = desfa("%2Fetc")
    assert strat_url == "url"
    assert rez_url == b"/etc"

def test_a4_citibil_si_straturi():
    assert citibil(b"Text valid 123!") is True
    assert citibil(b"\x00\x01\xff") is False
```

```
    val = "NTM1OTU3NTk="
    mesaj_final = val
    for _ in range(8):
        strat = ghici_strat(val)
        if strat == "necunoscut":
            break
        try:
            _, b = desfa(val)
            if not citibil(b):
                break
            val = b.decode("utf-8")
            mesaj_final = val
        except Exception:
            break
    assert mesaj_final == "SYWY"

def test_a5_spargere_xor():
    text_original = b"FLAG{test_xor}"
    cheie = 128
    criptat = bytes(b ^ cheie for b in text_original)

    gasit = None
    for k in range(256):
        candidat = bytes(b ^ k for b in criptat)
        if citibil(candidat) and candidat.startswith(b"FLAG{"):
            gasit = candidat.decode("utf-8")
            break
    assert gasit == "FLAG{test_xor}"

def test_a6_spargere_hash():
    parola = "secret"
    hash_tinta = hashlib.md5(parola.encode("utf-8")).hexdigest()
    assert len(hash_tinta) == 32

    lista_cuvinte = ["admin", "123456", "secret", "root"]
    gasit = None
    for cuv in lista_cuvinte:
        if hashlib.md5(cuv.encode("utf-8")).hexdigest() == hash_tinta:
            gasit = cuv
            break
    assert gasit == "secret"

def test_a7_fisa_json(tmp_path):
    strat, rez_bytes = desfa("U2FsdXQ=")
    date_salvare = {
        "strat": strat,
        "rezultat": rez_bytes.decode("utf-8")
    }
    cale_fisier = tmp_path / "fisa.json"
    with open(cale_fisier, "w", encoding="utf-8") as f:
        json.dump(date_salvare, f)

    with open(cale_fisier, "r", encoding="utf-8") as f:
        date_incarcate = json.load(f)

    assert date_incarcate["strat"] == "base64"
    assert date_incarcate["rezultat"] == "Salut"
```

Anexa C. Codul fișierului forensics.py

```
import os
import re
import zipfile
from pathlib import Path
from PIL import Image
from PIL.ExifTags import TAGS, GPSTAGS

# Localizează folderul Lab6 indiferent dacă scriptul e în Lab6/ sau
Lab6/arsenal/forensics/
BASE_LAB = Path(__file__).resolve().parent
if not (BASE_LAB / "probe").exists() and (BASE_LAB.parent.parent /
"probe").exists():
    BASE_LAB = BASE_LAB.parent.parent

os.chdir(BASE_LAB)

print("===== Exercitiul B1 =====")
def tip_real(cale):
    with open(cale, "rb") as f:
        cap = f.read(8)

    if cap.startswith(b"%PDF"):
        return "pdf"
    if cap.startswith(b"\x89PNG"):
        return "png"
    if cap.startswith(b"\xff\xd8"):
        return "jpeg"
    if cap.startswith(b"PK\x03\x04"):
        return "zip/docx"

    return cap.hex()

print("Tip probe/foto.jpg:", tip_real(BASE_LAB / "probe" / "foto.jpg"))
print("Tip probe/ascuns.png:", tip_real(BASE_LAB / "probe" / "ascuns.png"))

print("===== Exercitiul B2 =====")
date = open(BASE_LAB / "probe" / "ascuns.png", "rb").read()
for m in re.findall(rb"[ -~]{4,}", date):
    print(m.decode("latin1"))

print("===== Exercitiul B3 =====")
def dms_in_zecimal(dms, ref):
    grade = float(dms[0])
    minute = float(dms[1])
    secunde = float(dms[2])
    zecimal = grade + (minute / 60.0) + (secunde / 3600.0)
    if ref in ["S", "W"]:
        zecimal = -zecimal
    return zecimal

imagine = Image.open(BASE_LAB / "probe" / "foto.jpg")
exif = imagine._getexif() or {}

gps_info = {}
for id_tag, valoare in exif.items():
    nume_tag = TAGS.get(id_tag, id_tag)
    if nume_tag == "GPSInfo" or id_tag == 34853:
        for cheie_gps, val_gps in valoare.items():
            nume_subtag = GPSTAGS.get(cheie_gps, cheie_gps)
            gps_info[nume_subtag] = val_gps
```

```
if "GPSLatitude" in gps_info and "GPSLongitude" in gps_info:
    lat = dms_in_zecimal(gps_info["GPSLatitude"], gps_info.get("GPSLatitudeRef",
"N"))
    lon = dms_in_zecimal(gps_info["GPSLongitude"], gps_info.get("GPSLongitudeRef",
"E"))
    print(f"Latitudine: {lat:.6f}")
    print(f"Longitudine: {lon:.6f}")
    print(f"Harta: https://www.google.com/maps?q={lat},{lon}")
else:
    print("Nu au fost gasite coordonate GPS in metadate.")

print("===== Exercitiul B4 =====")
date = open(BASE_LAB / "probe" / "ascuns.png", "rb").read()
i = date.find(b"PK\x03\x04")

if i != -1:
    cale_zip = BASE_LAB / "gasit.zip"
    folder_extras = BASE_LAB / "extras_din_zip"

    with open(cale_zip, "wb") as f:
        f.write(date[i:])
    print(f"Arhiva extrasa cu succes in {cale_zip.name} de la octetul {i}")

    with zipfile.ZipFile(cale_zip, "r") as z:
        print("Fisiere gasite in arhiva:", z.namelist())
        z.extractall(folder_extras)
        print(f"Continutul a fost extras in directorul '{folder_extras.name}'")
else:
    print("Nu a fost gasita nicio semnatura ZIP in fisier.")
```

Anexa D. Codul fișierului stego.py

```
import os
from pathlib import Path
from PIL import Image

# Identifică folderul Lab6 indiferent dacă scriptul e în Lab6/ sau
Lab6/arsenal/forensics/
BASE_LAB = Path(__file__).resolve().parent
if not (BASE_LAB / "probe").exists() and (BASE_LAB.parent.parent /
"probe").exists():
    BASE_LAB = BASE_LAB.parent.parent

os.chdir(BASE_LAB)

def ascunde_mesaj(cale_intrare, cale_iesire, mesaj):
    img = Image.open(cale_intrare).convert("RGB")
    pixels = img.load()
    latime, inaltime = img.size

    # Adăugăm terminatorul nul \x00 pentru a ști când să ne oprim la extragere
    date = mesaj.encode("utf-8") + b"\x00"
    biti = "".join(f"{x:08b}" for x in date)

    idx_bit = 0
    total_biti = len(biti)

    for y in range(inaltime):
        for x in range(latime):
```

```
            if idx_bit >= total_biti:
                break
            r, g, b = pixels[x, y]
            canale = [r, g, b]
            for i in range(3):
                if idx_bit < total_biti:
                    canale[i] = (canale[i] & ~1) | int(biti[idx_bit])
                    idx_bit += 1
            pixels[x, y] = tuple(canale)
        if idx_bit >= total_biti:
            break

    img.save(cale_iesire)
    print(f"Mesaj ascuns salvat in {cale_iesire}")

def dezvaluie_mesaj(cale_imagine):
    img = Image.open(cale_imagine).convert("RGB")
    pixels = img.load()
    latime, inaltime = img.size

    octeti = bytearray()
    octet_curent = 0
    biti_in_octet = 0

    for y in range(inaltime):
        for x in range(latime):
            r, g, b = pixels[x, y]
            for canal in (r, g, b):
                bit = canal & 1
                octet_curent = (octet_curent << 1) | bit
                biti_in_octet += 1

                if biti_in_octet == 8:
                    if octet_curent == 0:
                        return octeti.decode("utf-8", errors="ignore")
                    octeti.append(octet_curent)
                    octet_curent = 0
                    biti_in_octet = 0

    return octeti.decode("utf-8", errors="ignore")

if __name__ == "__main__":
    cale_intrare = BASE_LAB / "probe" / "foto.jpg"
    cale_iesire = BASE_LAB / "stego.png"

    print("===== Exercițiul 2.9 (Steganografie LSB) =====")
    ascunde_mesaj(cale_intrare, cale_iesire, "FLAG{ascuns_in_pixeli}")
    rezultat = dezvaluie_mesaj(cale_iesire)
    print("Mesaj dezvăluit:", rezultat)
```

Anexa E. Conținutul fișierului arsenal/forensics/fisa.json

```
{
    "strat": "base64",
    "rezultat": "Salut"
}
```